

NLgov Assurance profile for OAuth 2.0



Logius Standard

Draft September 15, 2026

This version:

<https://logius-standaarden.github.io/OAuth-NL-profiel/>

Latest published version:

<https://gitdocumentatie.logius.nl/publicatie/api/oauth/>

Latest editor's draft:

<https://logius-standaarden.github.io/OAuth-NL-profiel/>

Previous version:

<https://gitdocumentatie.logius.nl/publicatie/api/oauth/v1.1.0/>

Editor:

Logius Standaarden ([Logius](#))

Authors:

Jaron Azaria ([Logius](#))

Martin Borgman ([Kadaster](#))

Marc Fleischeuers ([Kennisset](#))

Alexander Green ([Logius](#))

Peter Haasnoot ([Logius](#))

Heiko Hudig ([Logius](#))

Martin van der Plas ([Logius](#))

Stas Mironov ([Logius](#))

Leon van der Ree ([Logius](#))

Bob te Riele ([RvIG](#))

Remco Schaar ([Logius](#))

Frank Terpstra ([Geonovum](#))

Jan Jaap Zoutendijk ([Rijkswaterstaat](#))

Participate:

[GitHub Logius-standaarden/OAuth-NL-profiel](#)

[All issues](#)

[File an issue](#)

[Commit history](#)

[Pull requests](#)

This document is also available in these non-normative format: [PDF](#)



This document is licensed under

[Creative Commons Attribution 4.0 International Public License](https://creativecommons.org/licenses/by/4.0/)

Status of This Document

This is a draft that could be altered, removed or replaced by other documents. It is not a recommendation approved by TO.

Table of Contents

Status of This Document

Conformance

Requirements Notation and Conventions

Terminology

Conformance

Abstract

Dutch government Assurance profile for OAuth 2.0

Usecases

Introduction

Resource Server

Authorization Server

Client

Use case: Client credentials flow

Step 1. Client Authentication

Step 2. Access Token Response

Step 3. Resource interaction

Use case: Authorization code flow

Step 1. Authorization initiation

Step 2. Authorization Request

Step 3. User Authorization and consent

Step 4. Authorization Grant

Step 5. Access Token Request

Step 6. Access Token Response

Step 7. Resource interaction

Use case: Token exchange

Step 1: Token Exchange Request

Step 2: Access Token Response

Step 3: Resource Interaction

1. Client Profiles

1.1 Client Types

1.1.1 Full Client with User Delegation

1.1.2 Native Client with User Delegation

1.1.3 Direct Access Client

1.2 Connection to the Authorization Server

1.2.1 Requests to the Authorization Endpoint

1.2.2 Response from the Authorization Endpoint

1.2.3 Requests to the Token Endpoint

1.2.4 Client Keys

2. Authorization Server Profile

2.1 Connections with clients

2.1.1 Grant types

2.1.2 Client authentication

2.1.3 Dynamic Registration

2.1.4 Client Approval

2.1.5 Discovery

2.1.6 Revocation

2.1.7 PKCE

2.1.8 Redirect URIs

2.1.9 RefreshTokens

2.1.10 Token Response

2.2 Connections between authorization servers and protected resources

2.2.1 JSON Web Tokens (JWT)

2.2.2 Introspection

2.3 Response to Authorization Requests

2.4 Token Lifetimes

2.5 Scopes

2.5.1 Claims for Authorization Outside of Delegation Scenarios

3. Protected Resource Profile

3.1 Protecting Resources

3.2 Connections with Clients

3.3 Connections with Authorization Servers

4. Advanced OAuth Security Options

4.1 Pushed Authorization Requests (PAR)

4.2 Proof of Possession Tokens (PoP)

4.3 Rich Authorization Requests

5. Security Considerations

A. References

A.1 Normative references

A.2 Informative references

§ Conformance

As well as sections marked as non-normative, all authoring guidelines, diagrams, examples, and notes in this specification are non-normative. Everything else in this specification is normative.

The key words *MAY*, *MUST*, *MUST NOT*, *NOT RECOMMENDED*, *OPTIONAL*, *RECOMMENDED*, *REQUIRED*, *SHALL*, *SHALL NOT*, *SHOULD*, and *SHOULD NOT* in this document are to be interpreted as described in [BCP 14](#) [[RFC2119](#)] [[RFC8174](#)] when, and only when, they appear in all capitals, as shown here.

This profile is based upon the International Government Assurance Profile (iGov) for OAuth 2.0 [[iGOV.OAuth2](#)] as published by the [OpenID Foundation](#). It should be considered a fork of that profile, as the iGov profile is geared more towards the American situation; in the Netherlands the European Union context applies.

All requirements from [[iGOV.OAuth2](#)] apply to the corresponding NLgov roles (client, authorization server, protected resource) unless explicitly overridden or extended in this document. Readers *MUST* consult [[iGOV.OAuth2](#)] alongside this document.

§ Requirements Notation and Conventions

The key words "*MUST*", "*MUST NOT*", "*REQUIRED*", "*SHALL*", "*SHALL NOT*", "*SHOULD*", "*SHOULD NOT*", "*RECOMMENDED*", "*NOT RECOMMENDED*", "*MAY*", and "*OPTIONAL*" in this document are to be interpreted as described in [[rfc2119](#)] .

All uses of [JSON Web Signature (JWS)] [[rfc7515](#)] and [JSON Web Encryption (JWE)] [[rfc7516](#)] data structures in this specification utilize the JWS Compact Serialization or the JWE Compact Serialization; the JWS JSON Serialization and the JWE JSON Serialization are not used.

§ Terminology

This specification uses the terms "Access Token", "Authorization Code", "Authorization Endpoint", "Authorization Grant", "Authorization Server", "Client", "Client Authentication", "Client Identifier", "Client Secret", "Grant Type", "Protected Resource", "Redirection URI", "Refresh Token", "Resource Owner", "Resource Server", "Response Type", and "Token Endpoint" defined by [OAuth 2.0] [[rfc6749](#)], the terms "Claim Name", "Claim Value", and "JSON Web Token (JWT)" defined by [JSON Web Token (JWT)] [[rfc7519](#)], and the terms defined by [OpenID Connect Core 1.0] [[OpenID.Core](#)].

§ Conformance

Where [[iGOV.OAuth2](#)] refers to "iGov-compliant" components, this profile substitutes "NLgov-compliant". All conformance requirements of [[iGOV.OAuth2](#)] section 1.3 apply accordingly.

This document is an adaptation of the '[International Government Assurance Profile \(iGov\) for OAuth 2.0 - Draft 03](#)' (hereinafter: the iGov-profile) of the OpenID Foundation. This does not indicate an endorsement by the OpenID Foundation. In as far as the iGov-profile is incorporated in this document, the [OpenID Copyright License](#) applies.

Abstract

The OAuth 2.0 protocol framework defines a mechanism to allow a resource owner to delegate access to a protected resource for a client application.

This specification profiles the OAuth 2.0 protocol framework to increase baseline security, provide greater interoperability, and structure deployments in a manner specifically applicable, but not limited to consumer-to-government deployments in the Netherlands.

Organization / Committee	Version number	Official status	Date
Forum Standaardisatie	-	reported	16-03-2016
Working group	1.0	definitive approved version	15-07-2019
Forum Standaardisatie	1.0	'comply of explain' standard (mandatory open standard)	09-07-2020

Organization / Committee	Version number	Official status	Date
KP-API Working group	1.1.0-rc.1	working version / final draft by 'Working Group'	13-05-2024
OAuth-NL Working group	1.1.0-rc.2	proposed version - updated after public consultation	10-07-2024
MIDO programmeringstafel	1.1.0	definitive approved version	03-12-2024
Forum Standaardisatie	-	-	tbd

§ Dutch government Assurance profile for OAuth 2.0

This profile is based upon the [International Government Assurance Profile \(iGov\) for OAuth 2.0](#) as published by the [OpenID Foundation](#). It should be considered a fork of this profile as the iGov profile is geared more towards the American situation and in the Netherlands we have to deal with an European Union context.

We have added the chapter [Use cases](#) to illustrate the specific use case the iGov-NL profile is aimed at. Starting with chapter [Introduction](#) we follow the structure of the iGov profile. Where we do not use content from iGov we use ~~strikethrough~~ to indicate it is not part of iGov-NL.

iGov-NL : Additional content

Content added for the iGov-NL profile is indicated like this.

[The Governance of this standard](#) is described by the API-Standaarden beheermodel in a [separate repository](#) and published by Logius (api@logius.nl).

§ Usecases

There are two use cases: The *client credentials* flow and the *authorization code* flow. In two sections below we will elaborate on these, first we will introduce some common concepts.

§ Introduction

For the Client credentials flow and Authorization code flow usecases to work properly the following application building blocks need to be in place:

1. the **Resource Server** (usually described as the API)
2. the **Authorization Server**
3. the **Client** (application)

§ Resource Server

The service is provided by a public/governmental organization. Assumed is the Resource Server is known (by the Authorization Server) prior to actual authorization of the User. A Resource Server is assumed to possess a means for identification of the Resource Server and/or encrypted information, optionally by using a PKI certificate. Furthermore, a Resource Server is assumed to be provided over HTTP using TLS, other protocols are out of scope for this profile.

§ Authorization Server

An Authorization Server is available, operated by either an independent trusted third-party or the service provider itself. Only a single Authorization Server is in use. The Authorization Server is trusted by the Resource Server. The Authorization Server can identify and authorize the User. In case the User has no direct relationship to the Authorization Server, it can forward the User to an IDP trusted by both the Authorization Server as well as the User. Alternatively, the Authorization Server can otherwise identify and authorize the User and is trusted by that User.

§ Client

The User uses a client, which can be any arbitrary application decided upon by the User. Assumed is that the User trusts this client for interaction with the service. The authorization server has at least low trust in the client when the client is either public or semi-confidential. Assumptions is that the Client is aware of the specifications of the API and authorization is required. The Client is

either using a user-agent, typically a browser, or the relevant parts are integrated into the Client application.

Note: Web-applications by default use the system-browser on a User's device as user-agent. Typically a native application ("*mobile app*") either starts a system browser as user-agent or uses an *in-app* browser. See RFC 8252 for more information on implementation of native applications. Clients can also be 'machine clients' types.

§ Use case: Client credentials flow

The client credentials flow can be used in usecases where there is an Client to Resource server connection where no user information is needed by the resource server. Two examples are:

- An application does a system API call. For instance a ping service to see if an API is available. The user does not need to be logged in for this and there is no relation to the identity of the end user.
- A batch application processes a large number of transactions asynchronously at a later scheduled time. The original access_tokens of the preceding synchronous process is no longer available. The flow for such a machine to machine interaction is shown in the figure below.

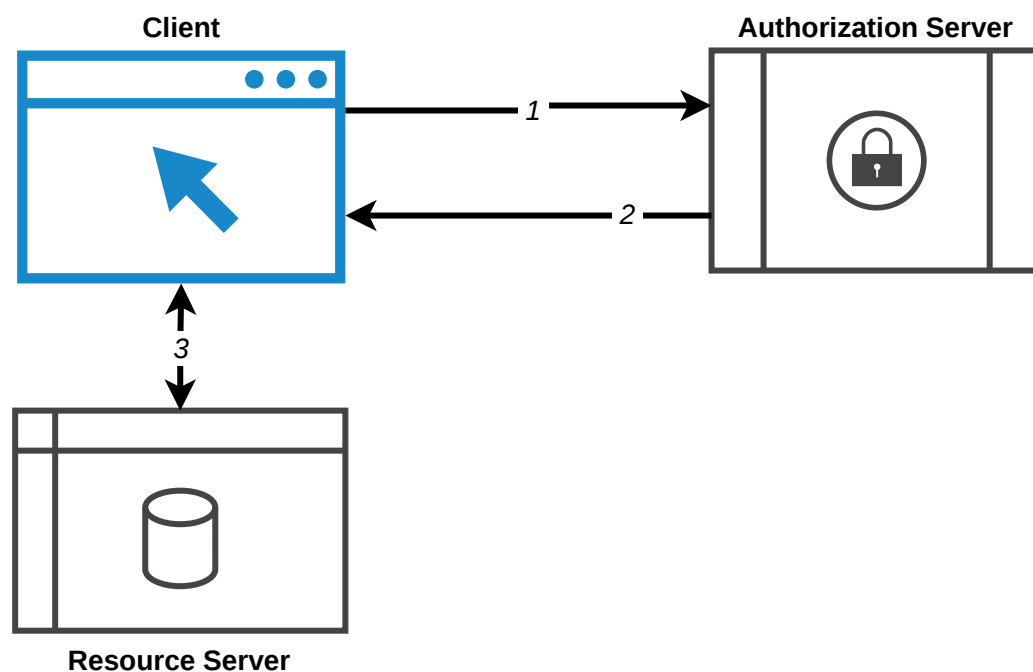


Figure 1 Use case Client credentials flow

§ Step 1. Client Authentication

Using the client credentials, the client sends a Authentication Request to the Authorization Server's token Endpoint. It does so using the Client authentication as pre-registered. The Authorization Server receives and validates the Authentication Request.

§ Step 2. Access Token Response

The Authorization Server authenticates the client and if valid responds to the client with an Access Token Response. The Authorization server issues an Access Token, specific to the requested authorization. The client receives the Access Token and can use the Access Token to send requests to the Service API.

§ Step 3. Resource interaction

The Client can now send (a) request(s) to the Service, on behalf of itself. It does so by sending requests to the Resource Server, along with the Access Token. The Resource Server uses the Access Token for its access control decision. The Resource Server responds based on these decisions to the Client. The contents and protocol of the Resource Request and Resource Response are out of scope of this profile.

Direct access clients that are using the client credentials grant type and are not using OpenIDConnect are also allowed to use an X.509 certificate to authenticate with the authorization server's token endpoint. This flow is compatible with OAuth 2.0 due to section 2.3.2 of [\[rfc6749\]](#).

§ Use case: Authorization code flow

In this use case a (public/governmental) service is offered via an API. The service will be consumed by the User using a client, that can be any arbitrary, non-trusted application. For provisioning the service, the service provider requires an identifier of the User. The identifier of the User can be either an arbitrary (self-registered) identifier or a formal identifier (citizen number or

other restricted, registered ID). Upon service provisioning, the service uses the identifier of the User for access control within the service.

A Client wishes to send a request to an API, on behalf of the User. The API requires to have a trusted identification and *authorization* of the User, before providing the Service. A Client has pre-registered with the Authorization Endpoint and has been assigned a client_id.

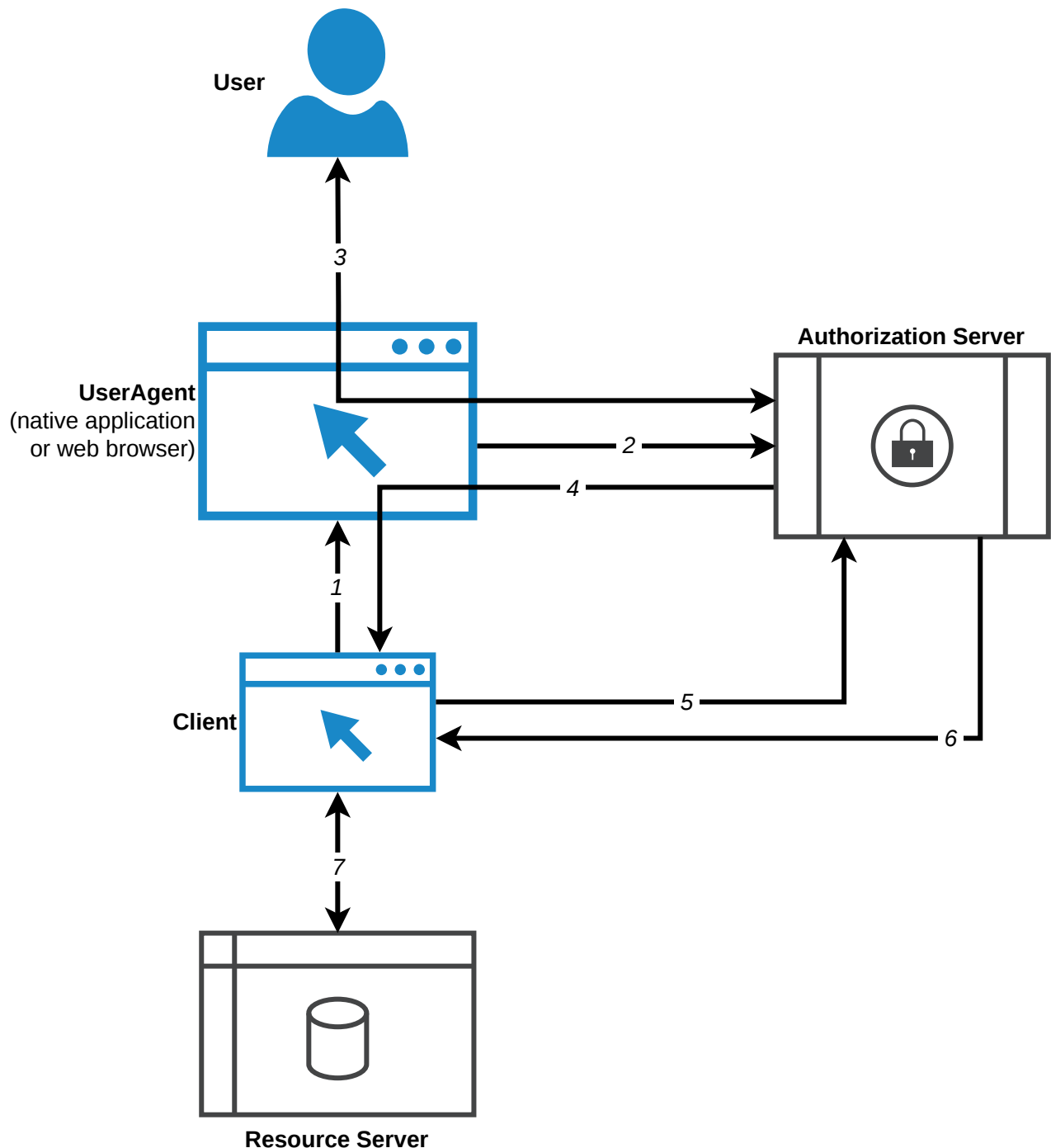


Figure 2 Use case Authorization code flow

The normal flow, that is without any error handling, is described below.

§ Step 1. Authorization initiation

As the client does not yet have a (valid) access token for this Service, it's first step is to obtain one. Therefore it sends an Authorization Request to the Authorization Server's Authorization Endpoint. It does so by redirecting / initiating the user-agent with the Authorization Request to the Authorization Endpoint. The Authorization request holds further details, as specified in this profile.

NOTE: Extra security consideration

When the Authorization Server supports [OAuth 2.0 Pushed Authorization Requests](#) (PAR), the client may first use PAR (or is required to use it, see `require_pushed_authorization_requests` in [Authorization Server Metadata](#)). The client can initiate the flow by pushing a POST request with the parameters to the `pushed_authorization_request_endpoint`. The Authorization Server responds to the client with a `request_uri` containing a reference. The client will then use this `request_uri` as the redirect.

§ Step 2. Authorization Request

The user-agent sends the Authorization request to the Authorization Endpoint. The Authorization Server receives and validates the request.

§ Step 3. User Authorization and consent

The Authorization Server identifies the Resource Owner (often, but not necessarily, the User) and obtains authorization and consent from the Resource Owner for using the client to access the Service. The method and means for identification, as well as how to obtain authorization and consent from the Resource Owner for the request, are implementation specific and explicitly left out of scope of this profile. Note that if the User and Resource Owner are one and the same, the Authorization Server will have to authenticate the User in order to reliably identify the User as Resource Owner before obtaining the authorization and consent.

§ Step 4. Authorization Grant

Note: applicable to the Authorization Code Flow only. The Authorization Server redirects the user-agent back to the Client, with a Authorization Response. This Authorization Response holds an Authorization Grant and is send to the `redirect_uri` endpoint from the Authorization request.

§ Step 5. Access Token Request

The Client receives the Authorization Response from the user-agent. Using the Authorization Grant from the response, the client sends a Token Request to the Authorization Server's token Endpoint. It does so using the Client authentication as pre-registered. The Authorization Server receives and validates the Token Request.

§ Step 6. Access Token Response

The Authorization Server responds to the client with an Access Token Response. This response contains an Access Token, specific to the requested authorization. The client receives and validates the Access Token and can use the Access Token to send requests to the Service API.

§ Step 7. Resource interaction

The Client can now send (a) request(s) to the Service, on behalf of its User. It does so by sending requests to the Resource Server, along with the Access Token. The Resource Server uses the Access Token for its access control decision and any customization of the service or data for the User, if applicable. The Resource Server responds based on these decisions to the Client. The Client can inform and interact with the User based on the information received from the Resource Server. The contents and protocol of the Resource Request and Resource Response are out of scope of this profile.

§ Use case: Token exchange

Token exchange is useful when the resource server requires a different token than the one(s) the client originally received during a prior interaction. To obtain this different token, the client can use token exchange [rfc8693]. It can be used for both impersonation and delegation, as specified in that RFC.

The flow for such a machine to machine interaction is shown in the figure below.

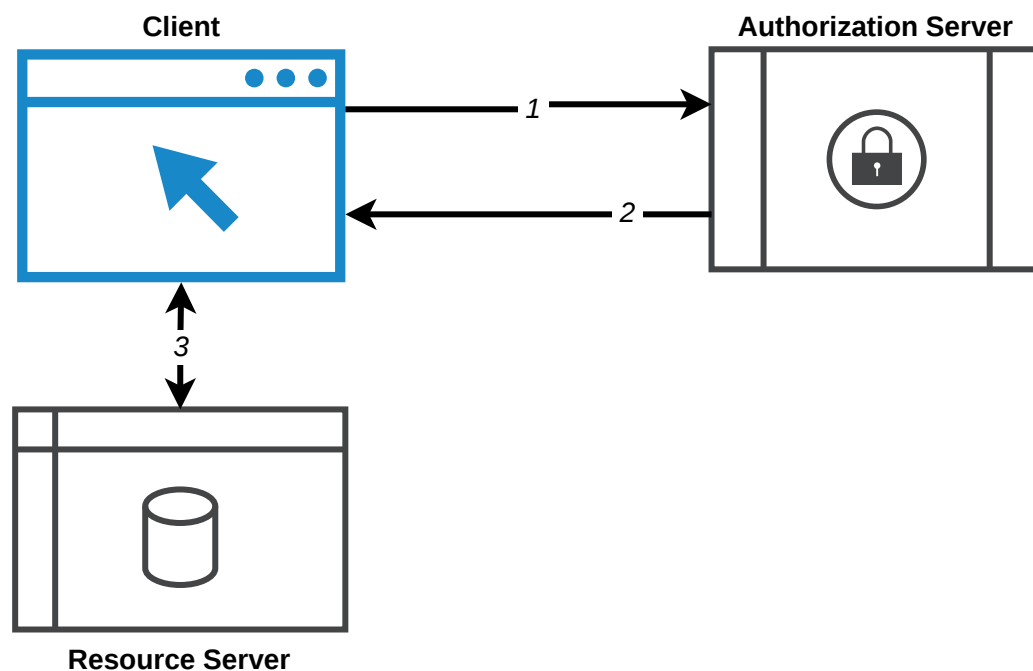


Figure 3 Use case token exchange

Note that the method by which the client received the original token(s)—either directly from an OAuth or SAML token server, or indirectly via other client applications—is not part of the token exchange process itself and, therefore, is not depicted in the diagram.

§ Step 1: Token Exchange Request

Using client credentials, the client sends a token exchange request to the Authorization Server's token endpoint. At a minimum, this request includes the subject token and the token exchange grant type. Optionally, the request may also include an actor token, audience, resource, and scopes.

§ Step 2: Access Token Response

The Authorization Server authenticates the client and validates the tokens and requested access, according to established policies. If valid, it issues a new token (typically an access token). This new token contains an "act" or "may_act" claim, linking the new token to the original subject token(s) as outlined in RFC 8693, as well as other claims relevant to the token type (e.g., an access token). The response may also include a refresh token, expiration details, and scopes. The client receives the new (access) token and can use it to make requests to the service API.

§ Step 3: Resource Interaction

The client can now use the new token to send requests to the service. These requests are directed to the Resource Server, which evaluates the access token as part of its access control decisions (e.g., by using the subject in the access token). The Resource Server responds based on these access control decisions. The specifics of the Resource Request and Resource Response are beyond the scope of this document.

§1. Client Profiles

§1.1 Client Types

§ 1.1.1 Full Client with User Delegation



In addition to `private_key_jwt`, the client authentication method `tls_client_auth` [rfc8705] *MAY* also be used.

§ 1.1.2 Native Client with User Delegation

§ 1.1.3 Direct Access Client

§ 1.2 Connection to the Authorization Server

§ 1.2.1 Requests to the Authorization Endpoint

§ 1.2.2 Response from the Authorization Endpoint

§ 1.2.3 Requests to the Token Endpoint

§ 1.2.4 Client Keys

Public clients *MUST* apply PKCE, as per [[rfc7636](#)]. As `code_challenge` the S256 method *MUST* be applied. Effectively this means that browser-based and native clients *MUST* include a cryptographic random `code_verifier` of at least 128 bits of entropy and the `code_challenge_method` with the value S256.

Request fields:

`client_id` : Mandatory. *MUST* have the value as obtained during registration.

`scope` : Optional.

`response_type` : Mandatory. *MUST* have value `code` for the Authorization Code Flow.

`redirect_uri` : Mandatory. *MUST* be an absolute HTTPS URL, pre-registered with the Authorization Server.

`state` : Mandatory. Do not use the SessionID secure cookie for this.

`code_challenge` : Mandatory when using a native app or browser-based app as user-agent.

`code_challenge_method` : Mandatory when `code_challenge` is used. *MUST* use the value S256.

When the Authorization Server supports OAuth 2.0 Pushed Authorization Requests (PAR) [[rfc9126](#)], the client *MAY* use PAR (or *MUST*, if `require_pushed_authorization_requests` is set in the Authorization Server Metadata). The client pushes the authorization parameters to the `pushed_authorization_request_endpoint` and uses the returned `request_uri` as the redirect.

Response parameters:

`code` : Mandatory. *MUST* be a cryptographic random value, using an unpredictable value with at least 128 bits of entropy.

`state` : Mandatory. *MUST* be a verbatim copy of the value of the `state` parameter in the Authorization Request.

When using the JWT assertion for client authentication, the assertion *MUST* use the claims as specified in [[rfc7521](#)] and [[rfc7523](#)].

In addition to `private_key_jwt`, the client authentication method `tls_client_auth` [[rfc8705](#)] *MAY* also be used. Private Key JWT is a method of client authentication where the client creates and signs a JWT using its own private key, as described in [[rfc7521](#)] and [[rfc7523](#)] and referenced by OpenID Connect and FAPI 2.0 Security Profile.

The Authorization Server *SHOULD* support the PS256 signing algorithm [[rfc7518](#)] for the signing of the `private_key_jwt`.

EXAMPLE 1

The following sample JWT contains the above claims and has been signed using the RS256 JWS algorithm and the client's own private key (with line breaks for display purposes only):

```
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.ew0KICAgImIzcyI6ICI1NWY5ZjU1OS0yNDk2LTQ5ZDQzNTFhNTg2Yjc0ODQiLA0KICAgImF1ZCI6ICJodHRwczovL2lkcc1wLmV4YW1wbGUuY29tLiwNCiAgICJpYXQiOiAxNDE4Njk4Nzg4LA0KICAgImV4cCI6IDE0MTg2OTg4NDgsDQogICAgaIjE0MTg2OTg3ODgvMTA3YzRkYTUxOTRkZjQ2M2U1MmI1Njg2NWM1YWYzNGU1NTk1Ig0KfJQGq3G20Ec2kUCQ8zVj7pqff87Sua5nktLIHj28l5on05VpsL4sRHIG0vrpo7X06jgtPWyLyo1TWHbtErQEGpmf7nKiNxVCXlGYJXSDJB6shP30fvdUc24urPJNUGBEDptIgT7-Lhf6BbNeOPRFDqQoLWqe7UxuI06dKX3SEQRmQcxYSIAfP7CQZ4WLuKXb6oEbaqz6gL4l6p83G7wTHszT-ZjKR38v4F_MnSrx8e0iIqgZwurW0RtetEWvyn0CJXk-p166T7qZR45xuCxg0otXY7GtgspMg0EKj3b_WpCiuNEwQ
```

This is sent in the request to the token endpoint as in the following example:

POST /token HTTP/1.1

Content-Type: application/x-www-form-urlencoded

User-Agent: Rack::OAuth2 (1.0.8.7) (2.5.3.2, ruby 2.1.3 (2014-09-19))

Accept: */*

Date: Tue, 16 Dec 2014 02:59:48 GMT

Content-Length: 884

Host: idp-p.example.com

grant_type=authorization_code

&code=sedaFh

&scope=openid+email

&client_id=55f9f559-2496-49d4-b6c3-351a586b7484

&redirect_uri=https%3A%2F%2Fclient.example.org%2Fcb

&client_assertion_type=urn%3Aietf%3Aparams%3Aoauth%3Aclient-assertion-

&client_assertion=eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.ew0KICAgImIzcyI6ICI1NWY5ZjU1OS0yNDk2LTQ5ZDQzNTFhNTg2Yjc0ODQiLA0KICAgImF1ZCI6ICJodHRwczovL2lkcc1wLmV4YW1wbGUuY29tL3Rva2VuIiwNCiAgICJpYXQiOiAxNDE4Njk4Nzg4LA0KICAgImV4cCI6IDE0MTg2OTg4NDgsDQogICAianRpIjogIjE0MTg2OTg3ODgvMTA3YzRkYTUxOTRkZjQ2M2U1MmI1Njg2NWM1NTk1Ig0KfQ.t-_gX8JQGq3G20Ec2kUCQ8zVj7pqff87Sua5nktLIHj28l5on05VpsL4sR7X06jgtPWy3iLXv3-NLyo1TWHbtErQEGpmf7nKiNxVCXlGYJXSDJB6shP30fvdUc24urPJIgT7-Lhf6BbwQNlMQubNeOPRFDqQoLWqe7UxuI06dKX3SEQRmQcxYSIAfP7CQZ4WLuKXb6L4l6p83G7wKGDeLET0THszT-ZjKR38v4F_MnSrx8e0iIqgZwurW0RtetEWvyn0CJXk-p16xuCxg0otXY603et4n77GtgspMg0EKj3b_WpCiuNEwQ

Where the Authorization Server, Resource Server, and client are not operated under the responsibility of the same organisation, each party *MUST* use PKIoverheid certificates with

OIN.

The PKIOverheid certificate *MUST* be included either as an x5c or as an x5u parameter, as per [rfc7517] sections 4.6 and 4.7. Parties *SHOULD* at least support the inclusion of the certificate as x5c for maximum interoperability. Parties *MAY* agree to use x5u, for instance for communication within specific environments.

§2. Authorization Server Profile

All servers *MUST* conform to applicable recommendations found in the Security Considerations sections of [rfc6749] and those found in the "OAuth Threat Model Document" [rfc6819] .

The authorization server *MUST* protect all communications to and from its OAuth endpoints using TLS.

§2.1 Connections with clients

§ 2.1.1 Grant types

The authorization server *MUST* support the `authorization_code` , and *MAY* support the `client_credentials` grant types as described in [Section 2](#). The authorization server *MUST* limit each registered client (identified by a client ID) to a single grant type only, since a single piece of software will be functioning at runtime in only one of the modes described in [Section 2](#). Clients that have multiple modes of operation *MUST* have a separate client ID for each mode.

iGov-NL : Additional content

Token exchange grant type [rfc8693] *SHOULD* be supported by the authorization server. This is used to translate a third party token (OAuth or SAML). For example, exchanging a DigiD or eHerkenning SAML token for an OAuth token.

When token exchange is not supported, SAML bearer grant [rfc7522] *MAY* be used as alternative.

§ 2.1.2 Client authentication

The authorization server *MUST* enforce client authentication as described above for the authorization code and client credentials grant types.

The authorization server *MUST* validate all redirect URIs for authorization code ~~and implicit grant types~~.

§ 2.1.3 Dynamic Registration

Dynamic Registration allows for authorized Clients to on-board programmatically without administrative intervention. This is particularly important in ecosystems with many potential Clients, including Mobile Apps acting as independent Clients. Authorization servers *MUST* support dynamic client registration, and clients *MAY* register using the [Dynamic Client Registration Protocol] [[rfc7591](#)] for authorization code grant types. Authorization servers *MAY* limit the scopes available to dynamically registered clients.

Authorization servers *MAY* protect their Dynamic Registration endpoints by requiring clients to present credentials that the authorization server would recognize as authorized participants. Authorization servers *MAY* accept signed software statements as described in [[rfc7591](#)] [[rfc7591](#)] issued to client software developers from a trusted registration entity. The software statement can be used to tie together many instances of the same client software that will be run, dynamically registered, and authorized separately at runtime. The software statement *MUST* include the following client metadata parameters:

redirect_uris

array of redirect URIs used by the client; subject to the requirements listed in [Section 2.2.1] (#redirect-uri)

grant_types

grant type used by the client; must be "authorization_code" or "client_credentials"

jwt_uri or jwks

client's public key in JWK Set format; if jwt_uri is used it *MUST* be reachable by the Authorization Server and point to the client's public key set

client_name

human-readable name of the client

client_uri

URL of a web page containing further information about the client

iGov-NL : Additional content

In this version of iGov-NL we recommend that the Authorization servers *SHOULD* support dynamic client registration. However depending on how the future authentication architecture of the Dutch government develops in regards to OAuth we may revisit this in a future revision. The current requirement fits an architecture where there is a limited number of widely used authorization servers. However if in practice we start seeing a very large number of authorization servers with limited use this requirement can become a recommendation in a future version of this profile. For these authorization servers with limited use we consider mandatory support for dynamic client registration a large burden.

§ 2.1.4 Client Approval

When prompting the end user with an interactive approval page, the authorization server *MUST* indicate to the user:

- Whether the client was dynamically registered, or else statically registered by a trusted administrator, or a public client.
- Whether the client is associated with a software statement, and in which case provide information about the trusted issuer of the software statement.
- What kind of access the client is requesting, including scope, protected resources (if applicable beyond scopes), and access duration.

For example, for native clients a message indicating a new App installation has been registered as a client can help users determine if this is the expected behaviour. This signal helps users protect themselves from potentially rogue clients.

§ 2.1.5 Discovery

The authorization server *MUST* provide an [OpenID Connect service discovery] [[OpenID.Discovery](#)] endpoint listing the components relevant to the OAuth protocol:

issuer

REQUIRED. The fully qualified issuer URL of the server

authorization_endpoint

REQUIRED. The fully qualified URL of the server's authorization endpoint defined by [OAuth 2.0] [[rfc6749](#)]

token_endpoint

REQUIRED. The fully qualified URL of the server's token endpoint defined by [OAuth 2.0] [[rfc6749](#)]

introspection_endpoint

OPTIONAL. The fully qualified URL of the server's introspection endpoint defined by [OAuth Token Introspection] [[rfc7662](#)]

revocation_endpoint

OPTIONAL. The fully qualified URL of the server's revocation endpoint defined by [OAuth 2.0 Token Revocation] [[rfc7009](#)]

jwks_uri

REQUIRED. The fully qualified URI of the server's public key in [JWK Set] [[rfc7517](#)] format

If the authorization server is also an OpenID Connect Provider, it *MUST* provide a discovery endpoint meeting the requirements listed in Section 3.6 of the iGov OpenID Connect profile.

EXAMPLE 2

The following example shows the JSON document found at a discovery endpoint for an authorization server:

iGov-NL : Additional content

Added `tls_client_auth`

```
{
  "request_parameter_supported": true,
  "registration_endpoint": "https://idp-p.example.com/register",
  "userinfo_signing_alg_values_supported": [
    "HS256", "HS384", "HS512", "RS256", "RS384", "RS512"
  ],
  "token_endpoint": "https://idp-p.example.com/token",
  "request_uri_parameter_supported": false,
  "request_object_encryption_enc_values_supported": [
    "A192CBC-HS384", "A192GCM", "A256CBC+HS512",
    "A128CBC+HS256", "A256CBC-HS512",
    "A128CBC-HS256", "A128GCM", "A256GCM"
  ],
  "token_endpoint_auth_methods_supported": [
    "private_key_jwt",
  ],
  "jwks_uri": "https://idp-p.example.com/jwk",
  "authorization_endpoint": "https://idp-p.example.com/authorize",
  "require_request_uri_registration": false,
  "introspection_endpoint": "https://idp-p.example.com/introspect",
  "request_object_encryption_alg_values_supported": [
    "RSA-OAEP", "?RSA1_5", "RSA-OAEP-256"
  ],
  "service_documentation": "https://idp-p.example.com/about",
  "response_types_supported": [
    "code", "token"
  ],
  "token_endpoint_auth_signing_alg_values_supported": [
    "HS256", "HS384", "HS512", "RS256", "RS384", "RS512"
  ],
  "revocation_endpoint": "https://idp-p.example.com/revoke",
  "request_object_signing_alg_values_supported": [
    "HS256", "HS384", "HS512", "RS256", "RS384", "RS512"
  ],
  "grant_types_supported": [
    "authorization_code",
    "client_credentials"
  ],
}
```

```
"scopes_supported": [
  "profile", "openid", "email", "address", "phone", "offline_access
],
"op_tos_uri": "https://idp-p.example.com/about",
"issuer": "https://idp-p.example.com/",
"op_policy_uri": "https://idp-p.example.com/about"
"tls_client_certificate_bound_access_tokens": "true"
"dpop_signing_alg_values_supported": ["PS256", "ES256"]
}
```

Clients and protected resources *SHOULD* cache this discovery information. It is *RECOMMENDED* that servers provide cache information through HTTP headers and make the cache valid for at least one week.

The server *MUST* provide its public key in JWK Set format. The key *MUST* contain the following fields:

kid

The key ID of the key pair used to sign this token

key

The key type

alg

The default algorithm used for this key

EXAMPLE 3

The following is an example of a 2048-bit RSA public key:

```
{
  "keys": [
    {
      "alg": "RS256",
      "e": "AQAB",
      "n": "o80vbR0ZfMhjZWfqwPUGNkcIeUcweFyzB2S2T-
hje83IOVct8gVg9FzvHPK1ReEW3-p7-A8GNcLAuFP_8jPhiL6LyJC3F10aV9KPQFF-
w6Eq6VtpEgYSfzvFegNiPtpMWd7C43EDwjQ-GrXMVCLrBYxZC-
P1ShyxVB0zeR_5MTC0JGiDTecr_2YT6o_3aE2SIJu4iNPgGh9MnyxdBo0Uf0TmrqEIa
bquXA1-V8iUihwfI8qjf3EujkYi7gXXeIIo4_gipQYNjr4DBNlE0__RI0kDU-
27mb6esswnP2WgHZQPsk779fTcNDBIcYgyLujlcUATEqfCaPDNp00J6AbY6w",
      "kty": "RSA",
      "kid": "rsa1"
    }
  ]
}
```

Clients and protected resources *SHOULD* cache this key. It is *RECOMMENDED* that servers provide cache information through HTTP headers and make the cache valid for at least one week.

iGov-NL : Additional content

iGov requires that the authorization server provides an OpenIDConnect service discovery endpoint. Recently OAuth 2.0 Authorization Server Metadata [\[rfc8414\]](#) has been finalized, this provide the same functionality in a more generic way and could replace this requirement in a future version of the iGov-NL profile.

§ 2.1.6 Revocation

Token revocation allows a client to signal to an authorization server that a given token will no longer be used.

An authorization server *MUST* revoke the token if the client requesting the revocation is the client to which the token was issued, the client has permission to revoke tokens, and the token is revocable.

A client *MUST* immediately discard the token and not use it again after revoking it.

§ 2.1.7 PKCE

An authorization server *MUST* support the Proof Key for Code Exchange (PKCE [[rfc7636](#)]) extension to the authorization code flow, including support for the S256 code challenge method. The authorization server *MUST NOT* allow an ~~iGov~~ iGov-NL client to use the plain code challenge method.

§ 2.1.8 Redirect URIs

The authorization server *MUST* compare a client's registered redirect URIs with the redirect URI presented during an authorization request using an exact string match.

§ 2.1.9 RefreshTokens

Authorization Servers *MAY* issue refresh tokens to clients under the following context:

Clients *MUST* be registered with the Authorization Server.

Clients *MUST* present a valid client_id. Confidential clients *MUST* present a signed client_assertion with their associated keypair.

Clients using the Direct Credentials method *MUST NOT* be issued refresh_tokens. These clients *MUST* present their client credentials with a new access_token request and the desired scope.

iGov-NL : Additional content

Refresh tokens for public clients must either be sender-constrained or one-time use. From [The OAuth 2.1 Authorization Framework: Refresh Token Grant](#)

§ 2.1.10 Token Response

iGov-NL : Additional content

The Token Response has the following contents:

access_token

Mandatory. Structured access token a.k.a. a JWT Bearer token. The JWT *MUST* be signed.

token_type

Mandatory. The type for a JWT Bearer token is Bearer, as per [[rfc6750](#)]

refresh_token

Under this profile, refresh tokens are supported.

expires_in

Optional. Lifetime of the access token, in seconds.

scope

Optional. Scope(s) of the access (token) granted, multiple scopes are separated by whitespace. The scope *MAY* be omitted if it is identical to the scope requested.

For best practices on token lifetime see section [Token Lifetimes](#).

§2.2 Connections between authorization servers and protected resources

Unlike the core OAuth protocol, the ~~iGov~~ iGov-NL profile defines interoperability requirements between authorization servers and resource servers.

[/]: Dit is §3.3 geworden in de nieuwe versie van iGov. De tekst is ook (qua zinsbouw) aangepast.

§ 2.2.1 JSON Web Tokens (JWT)

In order to facilitate interoperability with multiple protected resources, all ~~iGov~~ iGov-NL-compliant authorization servers *MUST* issue cryptographically signed tokens in the JSON Web Token (JWT) format as defined in [JSON Web Token \(JWT\) Profile for OAuth 2.0 Access Tokens](#) . The information carried in the JWT is intended to allow a protected resource to quickly test the integrity of the token without additional network calls, and to allow the protected resource to determine which authorization server issued the token. The protected resource *MAY* use the

authorization server token introspection service, which is in turn used for conveying service to retrieve additional security information about the token.

The server *MUST* issue tokens as JWTs with, at minimum, the following claims:

iss

The issuer URL of the server that issued the token

client_id

The client id of the client to whom this token was issued

exp

The expiration time (integer number of seconds since from 1970-01-01T00:00:00Z UTC), after which the token *MUST* be considered invalid

jti

A unique JWT Token ID value with at least 128 bits of entropy. This value *MUST NOT* be re-used in another token. Clients *MUST* check for reuse of jti values and reject all tokens issued with duplicate jti values.

sub

The identifier of the end-user that authorized this client, or the client id of a client acting on its own behalf (such as a bulk transfer). Since this information could potentially leak private user information, it should be used only when needed. End-user identifiers *SHOULD* be pairwise anonymous identifiers unless the audience requires otherwise.

aud

The audience of the token, an array containing the identifier(s) of protected resource(s) for which the token is valid, if this information is known. The aud claim may contain multiple values if the token is valid for multiple protected resources. Note that at runtime, the authorization server may not know the identifiers of all possible protected resources at which a token may be used.

iat

The "iat" (issued at) claim identifies the time at which the JWT was issued. This claim can be used to determine the age of the JWT. Its value *MUST* be a number containing a NumericDate value.

EXAMPLE 4

The following sample claim set illustrates the use of the required claims for an access token as defined in this profile; additional claims *MAY* be included in the claim set:

```
{
  "exp": 1418702388,
  "client_id": "55f9f559-2496-49d4-b6c3-351a586b7484",
  "iss": "https://idp-p.example.com/",
  "sub": "93ff28e3-3982-c34b-f2a4-98bb3d42b277",
  "aud": "api.example.com"
  "jti": "2402f87c-b6ce-45c4-95b0-7a3f2904997f",
  "iat": 1418698788,
  "acr": "myACR"
}
```

The access tokens *MUST* be signed with [JWS] [[RFC7515](#)] . If private_key_jwt is used, the authorization server *MUST* support the RS256 signature method for tokens and *MAY* use other asymmetric signing methods as defined in the [IANA JSON Web Signatures and Encryption Algorithms registry] [[JWS.JWE.Algs](#)] . The JWS header *MUST* contain the following fields:

iGov-NL : Additional content

In addition to above signing methods, the Authorization server *SHOULD* support PS256 signing algorithm [[rfc7518](#)] for the signing of the JWT Bearer Tokens.

kid

The key ID of the key pair used to sign this token

EXAMPLE 5

This example access token has been signed with the server's private key using RS256:

```
eyJhbGciOiJSUzI1NiJ9.eyJ0KICAgImV4cCI6IDE0MTg3MDIzODgsDQogICAiYXpwIjo
gIjU1ZjlmNTU5LTl0OTYtNDlkNC1iNmMzLTM1MWE1ODZiNzQ4NCIsDQogICAiaXNzIjo
gImh0dHBzOi8vaWRwLXAuZXhhbXBsZS5jb20vIiwNCiAgICJqdGkiOiAiMjY4NzQ4NzQ4
tYjZjZS00NWM0LTk1YjAtN2EzZjI5MDQ5OTdmIiwNCiAgICJpYXQiOiAxNDE4NjQ4NzQ4
4LA0KICAgImtpZCI6ICJyc2ExIiwNCiAgICJpYXQiOiAxNDE4NjQ4NzQ4NzQ4NzQ4NzQ4
EC0sXpHfnYYqb-CET9Ah5I0yXIDZ20qEyN98UydgSTpi01YJDDcZV4f4DgY0ZdG3yBW3
XqwUQwbGf7Gwza9Z4AdhjHjzQx-lChXAYfL1xz0SBDkVbJdDjtXbvaSIyF7ueWF3M1C
M70-GXuRY4iucKbuytz9e7eW4Egk4Aag13iTk9-l5V-tvL6dYu8IlR93GKsaKE8bng0
EZ04xcnq8s4V5Yykuc_NARBJENiKTJM8w3wh7xWP2gvMp39Y0XnuC0LyIx-J1ttX83xm
pXDalyY-4HT9XHT9V73fKF8rLWJu9grrA
```

Refresh tokens *SHOULD* be signed with [JWS] [[rfc7515](#)] using the same private key and contain the same set of claims as the access tokens.

The authorization server *MAY* encrypt access tokens and refresh tokens using [JWE] [[rfc7516](#)] . Encrypted access tokens *MUST* be encrypted using the public key of the protected resource. Encrypted refresh tokens *MUST* be encrypted using the authorization server's public key.

iGov-NL : Additional content

How to select or obtain the key to be used for encryption of an access token is out of scope of this profile. An early draft of "Resource Indicators for OAuth 2.0" exist and *COULD* be used. This draft describes usage of the resource parameter to indicate the applicable resource server.

In case the Authorization Server and Resource Server are not operated under responsibility of the same organization, the bearer token *MUST* be signed with the use of a PKIOverheid certificates with OIN.

If the bearer token is also encrypted the bearer token *MUST* be encrypted with the use of a PKIOverheid certificates with OIN.

§ 2.2.2 Introspection

Token introspection allows a protected resource to query the authorization server for metadata about a token. The protected resource makes a request like the following to the token introspection endpoint:

EXAMPLE 6

```
POST /introspect HTTP/1.1
User-Agent: Faraday v0.9.0
Content-Type: application/x-www-form-urlencoded
Accept-Encoding: gzip;q=1.0,deflate;q=0.6,identity;q=0.3
Accept: */*
Connection: close
Host: as-va.example.com
Content-Length: 1412

client_assertion=eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJpc3MiOiJhMmMzNjkxOS0wMWZmLTQ4MTAtYTgyOS00MDBmYWQzNTczNTEiLCJzdWIiOiJhMmMzNjkxOS0wMWZmLTQ4MTAtYTgyOS00MDBmYWQzNTczNTEiLCJhdWQiOiJodHRwczovL2FzLXZhLmV4YW1wbGUuY29tL3Rva2VuIiwiaWF0IjoxNDE4Njk4ODE0LCJleHAiOiJ0MTg2OTg4NzQsImp0aSI6IjE0MTg2OTg4MTQvZmNmNDQ2OGY2MDVjNjE1NjliOWYyNGY5ODJlMTZhZWY2OTU4In0.md7mFdNBaGhiJfE_pFkAAWA5S-JBvDw9Dk7p00JEWcL08JGgDFoi9UDbg3sHeA5DrrCYGC_zw7fCGc9ovpfMB7W6YN53hGU19LtzzFN3tv9FNRq4KIzhK15pns9jckKtui3HZ25L_B-BnxHe7xNo3kA1M-p51uYYIM0hw1SRi2pfwBKG508WntybLjuJ0R3X97zvqHn2Q7xdVyKlInyNPA8gIZK0HVssXxH0I6yRrAqvdMn_sneDTWPrqVpaR_c7rt8Ddd7drf_nTD1QxESVhYqKTax5Qfd-aq8gZz8gJCzS0yyfQh6DmdhmwgrSCCRC6BUQkeFNvjMVEYHQ9fr0NA
&client_assertion_type=urn%3Aietf%3Aparams%3Aoauth%3Aclient-assertion-
&client_id=a2c36919-01ff-4810-a829-400fad357351
&token=eyJhbGciOiJSUzI1NiJ9.eyJleHAiOiJ0MTg2MDI0MTQsImF1ZCI6WyJlbnZmYjcyYS05NzRmLTQwMDEtYmNiNy1lNjdjMmJjMDAzN2YiXSwiaXNzIjoiaHR0cHM6XC9cL2FzLXZhLmV4YW1wbGUuY29tXC8iLCJqdGkiOiIyMWIwNTk2ZC04NWQzLTQzN2MtYWQ4My1iM2YyY2UyNDcyNDQiLCJpYXQiOiJ0MTg2OTg4MTR9.FXDtEzDLbTHzFNroW7w27RLk5m0wprFfFH7h4bdfw5fR3pwiquejKmdfAbJvN3_yfAokBv06we5RARJUbdjmFFfRRW23cMbpGQCik7Nq4L012X_1J4Iew0QXXMLTyWQQ_BcBMjcw3MtPrY1Ao0cfB0JPx1k2jwRkYtyVTLWlff6S5gK-ciYf3b0bAdjoQEhd_IvssIPH3xuBJkmtkrTl fWR0Q0pdpeyVePkMSI28XZvDaGnxA4j7QI5loZYeyzGR9h70xQLVzqwwl1P0-F_0JaDFMJF01yl4IexfpoZZsB3HhF2vFdL6D_lLeHRy-H2g20zF59eMIsM_Ccs4G47862w
```

The client assertion parameter is structured as described in [Section 2.3.3](#).

The server responds to an introspection request with a JSON object representing the token containing the following fields as defined in the token introspection specification:

active

Boolean value indicating whether or not this token is currently active at this authorization server. Tokens that have been revoked, have expired, or were not issued by this authorization

server are considered non-active.

scope

Space-separated list of OAuth 2.0 scope values represented as a single string.

exp

Timestamp of when this token expires (integer number of seconds since from 1970-01-01T00:00:00Z UTC)

sub

An opaque string that uniquely identifies the user who authorized this token at this authorization server (if applicable). This string *MAY* be diversified per client.

client_id

An opaque string that uniquely identifies the OAuth 2.0 client that requested this token

EXAMPLE 7

The following example is a response from the introspection endpoint:

```
HTTP/1.1 200 OK
Date: Tue, 16 Dec 2014 03:00:14 GMT
Access-Control-Allow-Origin: *
Content-Type: application/json;charset=ISO-8859-1
Content-Language: en-US
Content-Length: 266
Connection: close
```

```
{
  "active": true,
  "scope": "file search visa",
  "exp": 1418702414,
  "sub": "{sub\u003d6WZQPpnQxV, iss\u003dhhttps://idp-p.example.com/}"
  "client_id": "e71fb72a-974f-4001-bcb7-e67c2bc0037f",
  "token_type": "Bearer"
}
```

The authorization server *MUST* require authentication for both the revocation and introspection endpoints as described in [Section 2.3.2](#) . Protected resources calling the introspection endpoint *MUST* use credentials distinct from any other OAuth client registered at the server.

A protected resource *MAY* cache the response from the introspection endpoint for a period of time no greater than half the lifetime of the token. A protected resource *MUST NOT* accept a token that is not active according to the response from the introspection endpoint.

§2.3 Response to Authorization Requests

The following data will be sent as an Authorization Response to the Authorization Code Flow as described above. The authentication response is sent via HTTP redirect to the redirect URI specified in the request.

The following fields *MUST* be included in the response:

state

REQUIRED. The value of the state parameter passed in in the authentication request. This value *MUST* match exactly.

code

REQUIRED. The authorization code, a random string issued by the IdP to be used in the request to the token endpoint.

PKCE parameters *MUST* be associated with the "code" as per Section 4.4 of [Proof Key for Code Exchange by OAuth Public Clients (PKCE)] [[rfc7636](#)]

EXAMPLE 8

The following is an example response:

```
https://client.example.org/cb?state=2ca3359dfbfd0&code=g0IFJ1hV6Rb1sXUd
```

§2.4 Token Lifetimes

This profile provides *RECOMMENDED* lifetimes for different types of tokens issued to different types of clients. Specific applications *MAY* issue tokens with different lifetimes. Any active token *MAY* be revoked at any time.

For clients using the authorization code grant type, access tokens *SHOULD* have a valid lifetime no greater than one hour, and refresh tokens (if issued) *SHOULD* have a valid lifetime no greater than twenty-four hours.

For public clients access tokens *SHOULD* have a valid lifetime no greater than fifteen minutes.

For clients using the client credentials grant type, access tokens *SHOULD* have a valid lifetime no greater than six hours.

§2.5 Scopes

Scopes define individual pieces of authority that can be requested by clients, granted by resource owners, and enforced by protected resources. Specific scope values will be highly dependent on the specific types of resources being protected in a given interface. OpenID Connect, for example, defines scope values to enable access to different attributes of user profiles.

Authorization servers *SHOULD* define and document default scope values that will be used if an authorization request does not specify a requested set of scopes.

To facilitate general use across a wide variety of protected resources, authorization servers *SHOULD* allow for the use of arbitrary scope values at runtime, such as allowing clients or protected resources to use arbitrary scope strings upon registration. Authorization servers *MAY* restrict certain scopes from use by dynamically registered systems or public clients.

§ 2.5.1 Claims for Authorization Outside of Delegation Scenarios

iGov-NL : Additional content

If there is a need to include resource owner memberships in roles and groups that are relevant to the resource being accessed, entitlements assigned to the resource owner for the targeted resource that the authorization server knows about. The authorization server *SHOULD* include such attributes as claims in a JWT access token as defined in section 2.2.3.1 of [[rfc9068](#)].

In cases where the default scopes provided by Authorization Server are not descriptive enough, one can make use of the `authorization_details` claim (see [4.3 Rich Authorization Requests](#)) which provide extra details and thus greater flexibility for the Resource Server to handle the request.

§3. Protected Resource Profile

§3.1 Protecting Resources

Protected Resources grant access to clients if they present a valid `access_token` with the appropriate `scope`. Resource servers trust the authorization server to authenticate the end user and

client appropriately for the importance, risk, and value level of the protected resource scope.

Protected resources that require a higher end-user authentication trust level to access certain resources *MUST* associate those resources with a unique scope.

Clients wishing access to these higher level resources *MUST* include the higher level scope in their authorization request to the authorization server.

Authorization servers *MUST* authenticate the end-user with the appropriate trust level before providing an `authorization_code` or associated `access_token` to the client.

Authorization servers *MUST* only grant access to higher level scope resources to clients that have permission to request these scope levels. Client authorization requests containing scopes that are outside their permission *MUST* be rejected.

Authorization servers *MAY* set the expiry time (`exp`) of `access_tokens` associated with higher level resources to be shorter than `access_tokens` for less sensitive resources.

Authorization servers *MAY* allow a `refresh_token` issued at a higher level to be used to obtain an `access_token` for a lower level resource scope with an extended expiry time. The client *MUST* request both the higher level scope and lower level scope in the original authorization request. This allows clients to continue accessing lower level resources after the higher level resource access has expired -- without requiring an additional user authentication/authorization.

EXAMPLE 9

A resource server has resources classified as "public" and "sensitive". "Sensitive" resources require the user to perform a two-factor authentication, and those access grants are short-lived: 15 minutes. For a client to obtain access to both "public" and "sensitive" resources, it makes an authorization request to the authorization server with `scope=public+sensitive`. The authorization server authenticates the end-user as required to meet the required trust level (two-factor authentication or some approved equivalent) and issues an `access_token` for the "public" and "sensitive" scopes with an expiry time of 15mins, and a `refresh_token` for the "public" scope with an expiry time of 24 hrs. The client can access both "public" and "sensitive" resources for 15mins with the `access_token`. When the `access_token` expires, the client will NOT be able to access "public" or "sensitive" resources any longer as the `access_token` has expired, and must obtain a new `access_token`. The client makes a access grant request (as described in [OAuth 2.0] [[rfc6749](#)] section 6) with the `refresh_token`, and the reduced scope of just "public". The token endpoint validates the `refresh_token`, and issues a new `access_token` for just the "public" scopewith an expiry time set to 24hrs. An access grant request for a new `access_token` with the "sensitive" scope would be rejected, and require the client to get the end-user to re-authenticate/authorize the "sensitive" scope request.

In this manner, protected resources and authorization servers work together to meet risk tolerance levels for sensitive resources and end-user authentication.

§3.2 Connections with Clients

A protected resource *MUST* accept bearer tokens passed in the authorization header as described in [rfc6750]. A protected resource *MAY* also accept bearer tokens passed in the form parameter ~~or query parameter~~ methods.

iGov-NL : Additional content

A Protected Resource under this profile *MUST NOT* accept access tokens passed using the query parameter method.

A Protected Resource under this profile *SHOULD* verify if the client is the Authorized party (azp) when client authentication is used. See section [Advanced OAuth Security Options](#) as well.

Protected resources *MUST* define and document which scopes are required for access to the resource.

§3.3 Connections with Authorization Servers

Protected resources *MUST* interpret access tokens using either JWT, token introspection, or a combination of the two.

The protected resource *MUST* check the aud (audience) claim, if it exists in the token, to ensure that it includes the protected resource's identifier. The protected resource *MUST* ensure that the rights associated with the token are sufficient to grant access to the resource. For example, this can be accomplished by querying the scopes and acr associated with the token from the authorization server's token introspection endpoint.

iGov-NL : Additional content

In case these (optional) attributes are already provided within the token, no introspection is needed. For further details we encourage to read the [NLgov Assurance profile for OpenID Connect 1.0](#) specification.

A protected resource *MUST* limit which authorization servers it will accept valid tokens from. A resource server *MAY* accomplish this using a whitelist of trusted servers, a dynamic policy engine, or other means.

§4. Advanced OAuth Security Options

The preceding portions of this OAuth profile provide a level of security adequate for a wide range of use cases, while still maintaining relative ease of implementation and usability for developers, system administrators, and end users. The following are some additional security measures that can be employed for use cases where elevated risks justify the use of additional controls at the expense of implementation effort and usability. This section also addresses future security capabilities, currently in the early draft stages, being added to the OAuth standard suite.

§4.1 Pushed Authorization Requests (PAR)



Traditionally, OAuth 2.0 authorization requests are sent via front-channel communication (e.g., browser redirects), which exposes sensitive parameters to potential tampering or interception. PAR [rfc9126] addresses these vulnerabilities by allowing clients to push authorization requests directly to the authorization server over a secure back-channel. [FAPI 2.0 Security Profile](#) also includes this feature as of version 2.0. Below are some of the issues it alleviates:

- **Lack of Integrity and Authenticity:** Authorization request parameters sent as URI query parameters are vulnerable to tampering. Attackers can modify values like scope or redirect_uri, potentially altering the context of transactions or access permissions. Such sensitive data in front-channel requests can be intercepted or phished, compromising client credentials or authorization codes. Attackers can exploit the request_uri parameter by injecting malicious URIs, leading to unauthorized access or token leakage.
- **Lack of Confidentiality:** Although HTTPS protects the authorization endpoint, request parameters pass through the user agent in the clear, risking exposure via browser logs, referrer headers, or other leaks. This is particularly problematic for sensitive data like personally identifiable information (PII).
- **Size Limitations:** Large authorization requests with fine-grained permissions can exceed URL size limits, causing processing errors.
- **Delayed Client Authentication:** Traditional flows delay client authentication until after user interaction, making it harder to detect and reject illegitimate requests early.

To combat this PAR allows clients to push authorization requests directly to the authorization server over a secure back-channel (HTTPS), effectively preventing tampering. For higher security, PAR can be combined with JWT-based Request Objects for cryptographic signing and optional encryption. Furthermore, by moving sensitive data from the front-channel (user agent) to the back-channel, PAR ensures that request parameters are not exposed to the browser or third parties, mitigating leakage risks. Also, PAR enables the authorization server to authenticate the client before any user interaction, allowing early detection and rejection of illegitimate requests, such as spoofing or tampering attempts.

§4.2 Proof of Possession Tokens (PoP)



Proof of possession can be implemented using various methods. One example is mutual TLS, where the client authenticates using a PKIoverheid certificate. The authorized party (azp) can then be verified against the client certificate.

As an alternative, the authorization server *MAY* include a `cnf` parameter in the JWT access token, as per [rfc7800]. The key referenced in `cnf` can be validated using client authentication such as `private_key_jwt` or `tls_client_auth` [rfc8705].

§4.3 Rich Authorization Requests



[OAuth 2.0 Rich Authorization Requests](#) is an extension that provides a way for clients to request and obtain fine-grained authorization from resource owners such as end users during the Authorization Code Flow. In traditional OAuth flows, clients typically request access to a set of scopes from a Resource Server. The Resource Owner then grants access to the resources to the client. However, this approach allows limited granular control over the access granted to a client and can lead to over-provisioning of access, which poses various security risks. With RAR, clients can pass an `authorization_details` claim with additional details that allow for more fine-grained authorization. This also allows for the Resource Server to implement fine-grained authorization for specific requests. Think of one-time payment approvals, document signing or requesting or approving (access to a) specific cases or documents.

According to the RFC, `authorization_details` requires just one field, `type`, which determines the allowable contents of the `authorization_details`. The value is unique for the described API in the context of the Authorization Server. The underlying RFC defines a set of common data fields that are designed to be usable across different types of APIs. Fields like

locations, datatypes, identifier and privileges can be added in the `authorization_details` parameter.

As mentioned in [RFC9396]:

In case of authorization requests as defined in [RFC6749], implementers *MAY* consider using pushed authorization requests [RFC9126] to improve the security, privacy, and reliability of the flow.

In case more data, or more recent data, is required for fine-grained authorization then one *MAY* include the OpenID AuthZEN standard in the `authorization_details`.

§5. Security Considerations



Response parameters:

`code` : Mandatory. *MUST* be a cryptographic random value, using an unpredictable value with at least 128 bits of entropy.

`state` : Mandatory. *MUST* be a verbatim copy of the value of the `state` parameter in the Authorization Request.

§A. References

§A.1 Normative references

[ietf-oauth-v2-1-10-refresh-token-grant]

The OAuth 2.1 Authorization Framework: Refresh Token Grant. Dick Hardt, Aaron Parecki, Torsten Lodderstedt. IETF. January 9, 2024. URL: <https://datatracker.ietf.org/doc/html/draft-ietf-oauth-v2-1-10#name-refresh-token-grant>

[JWS.JWE.Algs]

IANA JSON Web Signatures and Encryption Algorithms registry. Jim Schaad, Jeff Hodges, Joe Hildebrand, Sean Turner. IANA. URL: <https://www.iana.org/assignments/jose/jose.xhtml#web-signature-encryption-algorithms>

[NLgov.OpenID]

NLgov Assurance profile for OpenID Connect 1.0. Remco Schaar, Frank van Es, Joris Joosten, Jan Geert Koops. Logius. September 18, 2023. URL: <https://gitdocumentatie.logius.nl/publicatie/api/oidc/>

[OpenID.Discovery]

OpenID Connect Discovery 1.0. N. Sakimura, J. Bradley, M. Jones, E. Jay. OpenID foundation. November 8, 2014. URL: https://openid.net/specs/openid-connect-discovery-1_0.html

[OpenID.FAPI2.0]

FAPI 2.0 Security Profile. D. Fett, D. Tonge, J. Heenan. OpenID foundation. February 22, 2025. URL: https://openid.net/specs/fapi-security-profile-2_0-final.html

[RFC2119]

Key words for use in RFCs to Indicate Requirement Levels. S. Bradner. IETF. March 1997. Best Current Practice. URL: <https://www.rfc-editor.org/info/rfc2119/>

[rfc6749]

The OAuth 2.0 Authorization Framework. D. Hardt, Ed. IETF. October 2012. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc6749/>

[rfc6750]

The OAuth 2.0 Authorization Framework: Bearer Token Usage. M. Jones; D. Hardt. IETF. October 2012. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc6750/>

[rfc6819]

OAuth 2.0 Threat Model and Security Considerations. T. Lodderstedt, Ed.; M. McGloin; P. Hunt. IETF. January 2013. Informational. URL: <https://www.rfc-editor.org/info/rfc6819/>

[rfc7009]

OAuth 2.0 Token Revocation. T. Lodderstedt, Ed.; S. Dronia; M. Scurtescu. IETF. August 2013. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7009/>

[RFC7515]

JSON Web Signature (JWS). M. Jones; J. Bradley; N. Sakimura. IETF. May 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7515/>

[rfc7516]

JSON Web Encryption (JWE). M. Jones; J. Hildebrand. IETF. May 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7516/>

[rfc7517]

JSON Web Key (JWK). M. Jones. IETF. May 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7517/>

[rfc7518]

JSON Web Algorithms (JWA). M. Jones. IETF. May 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7518/>

[rfc7521]

Assertion Framework for OAuth 2.0 Client Authentication and Authorization Grants. B. Campbell; C. Mortimore; M. Jones; Y. Goland. IETF. May 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7521/>

[rfc7522]

Security Assertion Markup Language (SAML) 2.0 Profile for OAuth 2.0 Client Authentication and Authorization Grants. B. Campbell; C. Mortimore; M. Jones. IETF. May 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7522/>

[rfc7523]

JSON Web Token (JWT) Profile for OAuth 2.0 Client Authentication and Authorization Grants. M. Jones; B. Campbell; C. Mortimore. IETF. May 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7523/>

[rfc7591]

OAuth 2.0 Dynamic Client Registration Protocol. J. Richer, Ed.; M. Jones; J. Bradley; M. Machulak; P. Hunt. IETF. July 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7591/>

[rfc7636]

Proof Key for Code Exchange by OAuth Public Clients. N. Sakimura, Ed.; J. Bradley; N. Agarwal. IETF. September 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7636/>

[rfc7662]

OAuth 2.0 Token Introspection. J. Richer, Ed. IETF. October 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7662/>

[rfc7800]

Proof-of-Possession Key Semantics for JSON Web Tokens (JWTs). M. Jones; J. Bradley; H. Tschofenig. IETF. April 2016. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7800/>

[RFC8174]

Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. B. Leiba. IETF. May 2017. Best Current Practice. URL: <https://www.rfc-editor.org/info/rfc8174/>

[rfc8414]

OAuth 2.0 Authorization Server Metadata. M. Jones; N. Sakimura; J. Bradley. IETF. June 2018. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc8414/>

[rfc8693]

OAuth 2.0 Token Exchange. M. Jones; A. Nadalin; B. Campbell, Ed.; J. Bradley; C. Mortimore. IETF. January 2020. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc8693/>

[rfc8705]

OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens. B. Campbell; J. Bradley; N. Sakimura; T. Lodderstedt. IETF. February 2020. Proposed Standard.

URL: <https://www.rfc-editor.org/info/rfc8705/>

[rfc9068]

JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens. V. Bertocci. IETF. October 2021. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc9068/>

[rfc9126]

OAuth 2.0 Pushed Authorization Requests. T. Lodderstedt; B. Campbell; N. Sakimura; D. Tonge; F. Skokan. IETF. September 2021. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc9126/>

[RFC9396]

OAuth 2.0 Rich Authorization Requests. T. Lodderstedt; J. Richer; B. Campbell. IETF. May 2023. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc9396/>

§A.2 Informative references

[iGOV.OAuth2]

International Government Assurance Profile (iGov) for OAuth 2.0. J. Richer, M. Varley, P. Grassi. OpenID foundation. October 5, 2018. URL: https://openid.net/specs/openid-igov-oauth2-1_0-03.html

[OpenID.Core]

OpenID Connect Core 1.0. N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, C. Mortimore. OpenID foundation. November 8, 2014. URL: https://openid.net/specs/openid-connect-core-1_0.html

[rfc7519]

JSON Web Token (JWT). M. Jones; J. Bradley; N. Sakimura. IETF. May 2015. Proposed Standard. URL: <https://www.rfc-editor.org/info/rfc7519/>